

Problem 1

Let $G = (V, E)$ be a directed acyclic graph where each vertex $v \in V$ is associated with a number of stars between 1 and 5, denoted by $\star(v) \in \{1, 2, \dots, 5\}$, and let $s, t \in V$ be distinct vertices.² Describe and analyze an $O(V + E)$ -time algorithm to compute the shortest path from s to t in G with at least total 330 stars among the vertices in the path, or report that there is no such path. [Hint: Compute a topological ordering and use it to define a valid evaluation order.]

Solution: Following the hint, let $SIP(u, i)$ be the length of the shortest increasing path starting at $u \in V$ which considers the constraint that there must be at least 330 stars along the path. Since that G is a DAG, we can define the following recurrence:

$$SIP((u, i)) = \begin{cases} 0 & i \geq 330 \\ \infty & \text{if no out edge connects in } E \\ 1 + \max\{SIP((u, i)) \mid (u, i) \rightarrow (v, i + \star(v)), uv \in E'\} & \text{if } v \neq s \end{cases}$$

This defines a topological order. Also note that i solely depends on u , so we can index based on such ordering with a 1D array. Each element of the array gives us $SIP(u, i)$, hence our evaluation order should be in reverse topological order.

Solving the original problem just requires taking the min of our memoization array, which is $O(V) = O(V)$.

For runtime, computing the topological order requires running DFS, which takes $O(V + E)$. Each summation takes $O(1 + \sum_{u \in V} \text{outdegree}(u)) = O(E)$. Hence our total runtime is just $O(V + E)$.

Problem 2

Let $G = (V, E)$ be a directed graph where each edge is colored grey or Duke blue, denoted by $c(e) \in \{g, b\}$, and let s, t be distinct vertices. Describe a $O(V + E)$ -time algorithm to determine if there is a walk in G from s to t where no three consecutive edges have the same color. For example, if the walk has two grey edges in a row, then either the walk must end or the next edge after those two grey edges must be blue. Justify its correctness and analyze its runtime complexity.

Solution: Construct simple directed graph $G' = (V', E')$

$$\begin{aligned} V' &= V \times \{b, g\} \\ E_{bb} &= \{(u, b) \rightarrow (v, b) \rightarrow (w, g) \mid uv, vw \in E, c(uv) = c(vw) = b \in \{g, b\}\} \\ E_{bg} &= \{(u, b) \rightarrow (v, g) \rightarrow (w, g) \mid uv, vw \in E, c(uv) = b, c(vw) = g \in \{g, b\}\} \\ &\quad \cup \{(u, b) \rightarrow (v, g) \rightarrow (w, b) \mid uv, vw \in E, c(uv) = b, c(vw) = g \in \{g, b\}\} \\ E_{gg} &= \{(u, g) \rightarrow (v, g) \rightarrow (w, b) \mid uv, vw \in E, c(uv) = c(vw) = g \in \{g, b\}\} \\ E_{gb} &= \{(u, g) \rightarrow (v, b) \rightarrow (w, g) \mid uv, vw \in E, c(uv) = g, c(vw) = b \in \{g, b\}\} \\ &\quad \cup \{(u, g) \rightarrow (v, b) \rightarrow (w, b) \mid uv, vw \in E, c(uv) = c(vw) = b \in \{g, b\}\} \\ E'_S &= \{S \rightarrow (s, b), S \rightarrow (s, g)\} \\ E'_T &= \{(t, b) \rightarrow T, (t, g) \rightarrow T\} \\ E' &= E'_S \cup E'_T \cup E_{gb} \cup E_{gg} \cup E_{bg} \cup E_{bb} \end{aligned}$$

We note that our resulting graph is still a simple graph, the union of edges for E_{bg} and E_{gb} just notationally means there will be two out edges coming from (v, g) and (v, b) respectively. See that

by construction, there are no three consecutive edges with the same color with any walk in G' . See that we reduced our problem to one of reachability, i.e. can we reach T from S , where S is the source and T is the sink. Hence, we can just run DFS on G' starting from S and see if T is reached to solve our original problem.

Runtime: There are $2V + 2$ vertices and at most $4E + 2V$ edges in G' . Constructing the graph takes $O(V + E)$. The DFS on G' runs in $O(V' + E') = O(V + E)$ time. Therefore, our overall runtime is $O(V + E)$.

Problem 3

Alex is shopping for a new house in another city. Its road network is represented by an undirected graph $G = (V, E)$ with positive edge weights given by $w : E \rightarrow \mathbb{R}^+$, where the vertices are points-of-interest, and edges are direct road segments between them, and $w(e)$ for an edge e is the amount of time in minutes it takes to travel down (following the speed limit). (There are no one-way streets, so modeling the edges as undirected suffices.)

Let $H \subset V$ be a subset of houses for sale, let $D \subset V$ be the subset of daycares, and let $p \in V$ be the parking lot for his new job's building. Every weekday, Alex has to drop his son off at a daycare on his way to work, then pick his son up (at the same daycare) on his way home after work. Also interested in minimizing his commute, Alex wants to find the house $h \in H$ with the shortest total driving time each day. That is, the shortest drive from a house $h \in H$ to a daycare $d \in D$, then to p , then back to d , and finally back to h .

Describe an $O(E + V \log V)$ -time algorithm to find the desired house for Alex. Justify its correctness and analyze its runtime complexity. [Hint: First, would Alex use different routes to and from work? Second, ideas from the graph reductions in Discussion 5 may be useful here.]

Solution: We construct a new graph $G' = (V', E')$:

$$\begin{aligned} V' &= \{v_0, v_1 \mid v \in V\} \\ E' &= \{u_i \rightarrow v_i \mid u \rightarrow v \in E, i \in 0, 1\} \cup \\ &\quad \{d_0 \rightarrow v_1 \mid d \rightarrow v \in E \wedge d \in D\} \end{aligned}$$

The weight of an edge $u_i \rightarrow v_i$ in G' is the same as the corresponding edge in G , $w(u \rightarrow v)$. We create two copies v_0, v_1 of each vertex, where the subscript is 0 to denote the path to arbitrary node in H must include at least one node in D , and the subscript 1 denotes the path to any node in H does not need to include nodes in D . For each edge $u \rightarrow v \in E$, we add two copies to our graph, $u_0 \rightarrow v_0$ and $u_1 \rightarrow v_1$. We also add edges from $d \in D$ to each node $h \in H$ for which h is an out-neighbor of d in G .

By construction of G' , there is a shortest walk in G from p to $h \in H$ that visits $d \in D$. The problem can be converted to finding a node $h \in H$ so that the walk in G from p to h that visits $d \in D$ is the shortest.

Runtime: Constructing G' takes $O(1)$ time per vertex and edge. There are at most $2 * V$ vertices and at most $3E$ edges. Running Dijkstra's on G' takes $O(E' + V' \log(V')) = O(E + V \log(V))$ time, and thus the overall runtime is $O(E + V \log V)$.

Problem 4

There are n airports, each given an index from 1 to n . There are m flights between pairs of airports each day, where at most F flights depart from each airport. Each j -th flight is represented by a

4-tuple (d_j, a_j, t_j, ℓ_j) , where:

- $d_j \in \{1, \dots, n\}$ is the airport from the flight departs,
- $a_j \in \{1, \dots, n\}$ is the airport at which the flight arrives,
- $t_j \in \{0, 1, \dots, 1439\}$ is the number of minutes after midnight that the flight departs from airport d_j (e.g., $t_j = 1013$ corresponds to 4:53 PM and $t_j = 1439$ corresponds to 11:59 PM), and
- $\ell_j \in \{0, 1, \dots, 1439\}$ is the number of minutes after midnight that the flight arrives at airport a on the same day.

Note that, by definition, $0 \leq t_j < \ell_j < 1440$, and no flight is midair at midnight; all flights land on the same day that they depart. For example, $(9, 3, 653, 1136)$ represents a flight from airport 9 to airport 3 that departs at 10:53 AM and lands at 6:56 PM.

Suppose you wish to find the earliest time that you can arrive at airport n , assuming you begin at airport 1 at midnight (time 0), by taking a sequence of flights (or possibly a direct one). As in real life, you can only take a flight that departs from an airport i if you are presently at that airport before the flight departs. Describe an algorithm to find this earliest arrival time, whose input is the number of airports, n , and the m tuples representing flights, where at most F flights depart from each airport. Your algorithm should be as fast as you can achieve in terms of n , m , and F .³ Justify its correctness and analyze its runtime complexity.

To think about later: (a) Suppose you wish to minimize the total time spent in-flight, not counting time at airports, to travel from airport 1 to n . (b) Suppose flights come with a 5th value, the price to take the flight, and now you wish to minimize the total cost to travel between airports 1 and n .

Solution: Construct a directed weighted graph $G = (V, E)$:

$$\begin{aligned}
 V &= \{v \mid v = (d_v, a_v, t_v, l_v)\} \cup \text{source} = (0, 1, 0, 0) \\
 E &= \{(d_v, a_v, t_v, l_v) \rightarrow (d_u, a_u, t_u, l_u) \mid u, v \in V, a_v = d_u, l_v < t_u\} \\
 &\quad \cup \{(0, 0, 0, 0) \rightarrow (d_v, a_v, t_v, l_v) \mid d_v = 0\} \\
 w(E) &= l_u - l_v
 \end{aligned}$$

In graph G , vertices are constructed from the tuples that represents all m flights, with a source node representing the beginning at midnight and is linked to all flights taking off from airport 1. Edges are pointing from flight i to flight j if i lands at the airport where j departs and i lands before j departs, and the weights represent duration of flight j + waiting time between flight i and j . Weight of edges pointing from source node represents time spent before the first flight lands. Then, if we traverse through graph G , we are taking the flights in sequence. Our goal is to find a sequence of flights that travel from airport 1 to airport n with the earliest time landing at n . Thus, we use Dijkstra's algorithm to find the minimum weighted path from source node to all nodes with $l_v = n$.

Runtime: Building graph G takes $O(V + E)$ time to traverse through all flights and calculate all edges weights. We have m flights and at most F edges for all flights, so the runtime is $O(m + F * n)$. Running Dijkstra's algorithm takes $O(E + V \log(V)) = O(F * n + m \log(m))$ time. The total runtime is $O(F * n + m \log(m))$.